Exercise 2: Interest Points and Descriptors

In this exercise, you will implement an interest point detector, the Harris corner detector. You will then compute descriptors at each of the interest points. These descriptors will be used to match similar interest points from consecutive image frames with each other. An example of a good matching result can be seen in Figure 3 below.



Figure 1: Result of the image stitching using Harris corners.

The Harris Corner detector and the simple descriptors are sufficient for problems such as stitching a panorama image (see Figure 1) where the images are very similar. As the next step, we want to do object detection in images. For this a more sophisticated feature detector is required. You will use the SIFT feature detector of the OpenCV. You will first match the features and then apply the RANSAC algorithm to extract transformation matrices. An example of the detected object is shown in Figure 2.

Part 1: Implementation (12 points)

You should write code so that it can be understood by someone else (the tutors, for example). Comment sections and lines which might not be easy to understand. If you've thought about a line or section for more than a couple of minutes, it probably warrants a comment. Use descriptive variable names and try to follow the Python style guide. At the end of this section you can find tipps and useful commands.

Your code should not take too long to run. For the first few exercises, anything more than a few seconds likely means you are doing something inefficient. Points may be



Figure 2: A bounding box is drawn around the detected object using the estimated transformation matrix.

deducted for slow code, even if your solution is correct. You should not need to import any additional libraries. If you think an additional library is necessary, first ask in the forum. It is difficult to name each function you are not allowed to use because especially OpenCV is very powerful and has many of the algorithms we will implement in this course already integrated. As a general guideline, if a task can be solved with one function call, you probably should not use it. If you are unsure, please ask in the TUWEL forum.

1. Harris corner detector (3 points)

In the file **harris_corner.py** you should write code for the Harris corner detector presented in the lecture. This function is called from the **main_harris.py** script.

The input parameters are a grayscale image with values in the range of [0,1] and the following parameters:

- `k`: Coefficient of the Harris formula
- `threshold`: Minimum value of the Harris response (R) for a corner to be detected.
- `sigma1`: Sigma of Gaussian filter used in the first step of the algorithm
- `sigma2`: Sigma of Gaussian filter used after the calculation of the derivatives in x and y direction (should be larger than `sigma1`)

The basic steps you will have to implement are:

1. Compute the horizontal and vertical derivatives of the image ($\mathbf{I}_x$ and $\mathbf{I}_y$) by convolving the image $\mathbf{I}$ with derivatives of Gaussians in x- and y-direction. You can create the Gaussians as in exercise 1, using `sigma1` or you can use these commands:

```
# Produces an array with (kernel_width x 1)
kernel = cv2.getGaussianKernel(kernel_width, sigma)

# Produces an Gaussian kernel of shape (kernel_width x kernel_width)
gauss = np.outer(kernel, kernel.transpose())
```

   The width of the filter can be calculated like you did in exercise 1. To get a Gaussian derivative you can filter a Gaussian kernel with the small filter [-1 0 1], or use the `np.gradient` command.

2. Compute the Harris feature value for each pixel and normalize the values to have a maximum value of 1. Use another Gaussian (with `sigma2`) as the weighting kernel.

3. Apply non-maximum suppression and threshold the feature values. You can adapt your non-max suppression from the previous exercise or use the code snippet below. We also provide the `non_max()` function with the code from below in **helper_functions.py** for convenience.

```
# Initialize a 3x3 kernel with ones and a zero in the middle
kernel = np.ones(3, 3, dtype=np.uint8)
kernel[1, 1] = 0

# Apply the dilate morphology transformation.
# For details see https://docs.opencv.org/4.x/d9/d61/
    tutorial_py_morphological_ops.html
dilation = cv2.dilate(input_array, kernel)
local_maxima = input_array > dilation
```

Complete the code in **harris_corner.py**. To test your implementation you can call the script `main_harris.py`.

2. Descriptors (2 points)

Once the corner detector is complete, we want to match corners in similar images to find correspondences. To be able to do that, you first have to implement a feature descriptor characterising each detected corner. For the basic descriptor, take the image intensity values in a patch around a corner and concatenate them to get a large feature vector. The size of the patch is dependent on the parameter `patch_size`. You should filter out matches where part of the patch would be outside of the image. You can also experiment with adapting your feature vector (e.g., normalizing, sorting). If you do so, make sure to add comments about what you did.

Complete the code for the function `compute_descriptors` in **descriptors.py**.



Figure 3: Final result of matching Harris corners after filtering.

3. Matching interest points and filtering (1 point)

With the descriptors, we can match similar interest points across different images. We will use the `cv2.FlannBasedMatcher` which is already called in the **main_harris.py** file. This function calculates the Euclidean distance between all descriptors $\{\mathbf{x}_{A1}, \mathbf{x}_{A2}, \ldots, \mathbf{x}_{Am}\}$ of image $\mathbf{A}$ to each descriptor $\{\mathbf{x}_{B1}, \mathbf{x}_{B2}, \ldots, \mathbf{x}_{Bn}\}$ of image $\mathbf{B}$. The k-nearest neighbors (smallest distance) to a descriptor $\mathbf{x}_{Ai}$ in image $\mathbf{B}$ are stored as matches.

Some descriptors can be very ambiguous and cause false matches. Before we draw the matches and process them further, we want to filter out bad matches. The author of the SIFT paper (David G. Lowe) suggests to apply this heuristic: Consider the two "closest" matches $\mathbf{x}_{B1}$ and $\mathbf{x}_{B2}$ to $\mathbf{x}_{A1}$ with distances d_1 and d_2. The descriptor $\mathbf{x}_{B1}$ should only be considered as the unique best match for $\mathbf{x}_A$ if $\frac{d_1}{d_2}<0.8$. That way, many ambiguous and probably wrong matches will be discarded. After filtering, we can draw the matches using `cv2.drawMatches`. A possible result is shown in Figure 3.

Complete the code of the function `filter_matches` in **helper_functions.py**.

4. Estimate Projection Matrix (2 points)

As a next step, we want to detect an object in an image. We will use the SIFT feature detector provided by the OpenCV library. Compared to the feature descriptors you implemented before with the Harris Corner detector, SIFT is invariant to image scale and rotation. From the matches, we want to calculate the projective transformation matrix (homography) $\tilde{\mathbf{H}} \in \mathbb{R}^{3\times3}$ that projects the pixels from the object space into the scene space. With this matrix, we can draw a border around the detected object. In this part, you will implement the RANSAC algorithm to robustly calculate the transformation matrix. You can see the final result in Figure 2.

We will first derive the equations to calculate the projective transformation matrix. It uses homogeneous coordinates and can be written as

$$\tilde{\mathbf{x}} = \tilde{\mathbf{H}}\tilde{\mathbf{x}} \quad \text{with} \quad \tilde{\mathbf{H}} \in \mathbb{R}^{3\times 3},$$
$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

To obtain the transformed point, we have to eliminate the variable w which results in

$$x' = \frac{h_{00}x + h_{01}y + h_{02}}{h_{20}x + h_{21}y + h_{22}} \quad \text{and} \quad y' = \frac{h_{10}x + h_{11}y + h_{12}}{h_{20}x + h_{21}y + h_{22}}.$$

These equations can be rewritten as a system of linear equations in matrix form $\mathbf{Ax} = \mathbf{b}$

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -x'x & -x'y & -x' \\ 0 & 0 & 0 & x & y & 1 & -y'x & -y'y & -y' \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Feature matching produces multiple point correspondences that can easily be added to the equations

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1x_1 & -x'_1y_1 & -x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1x_1 & -y'_1y_1 & -y'_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_nx_n & -x'_ny_n & -x'_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_nx_n & -y'_ny_n & -y'_n \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

Before solving this system of equation, we can simplify it. The matrix $\tilde{\mathbf{H}}$ can be multiplied with an arbitrary non-zero scale factor without changing the projective transformation because in the homogenous representation of a point, only the ratio of the elements is significant. In other words, the projective transformation has eight degrees of freedom.

Therefore, we can choose one element of $\tilde{\mathbf{H}}$ arbitrarily. A common choice to normalize the matrix is to set $h_{22} = 1$ which results in the equation

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1x_1 & -x'_1y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1x_1 & -y'_1y_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 & -x'_nx_n & -x'_ny_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y'_nx_n & -y'_ny_n \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}.$$

We need at least four point correspondences (matches) which equals to eight scalar values to solve this equation. If we provide more point correspondences, we can apply the least-squares algorithm to find a solution. You should implement the function `find_homography_leastsquares()` which can deal with 4 or more point correspondences and applies the least-squares algorithm to calculate the projection matrix $\tilde{\mathbf{H}}$. To get the least-squares solution of this system, NumPy offers the function `numpy.linalg.lstsq()`. To debug this function, the function `debug_homography()` is provided in **helper_functions.py**. This function creates a rectangle, applies a random perspective transformation and returns the resulting image and the corners as matching keypoints in the object space and the scene space. It is already called in **main_sift.py** and passed to `find_homography_leastsquares()` if `debug = True`.

Complete the code for the function `find_homography_leastsquares()` in the file **find_homography.py**.

5. RANSAC (4 points)

The RANSAC (Random Sample Consensus) algorithm is more robust against outliers than the least-squares algorithm by itself. Therefore, it is suitable in the object detection scenario with multiple wrong matches. The algorithm is defined in the lecture slides (lecture 4). Implement the algorithm inside the `find_homography_ransac()`. The steps you will have to implement are:

1. We filter the matches using the function `filter_matches` in **main_sift.py**.
2. Now you can start with the RANSAC algorithm in `find_homography_ransac()`. Sample four matches from the filtered matches randomly. Use your `find_homography_leastsquares()` function with the 4 matches to calculate the perspective projection matrix $\tilde{\mathbf{H}}$. Apply this transformation to all points in the source space and calculate the Euclidean distances of all the transformed points to the respective points in the target space. From these distances, you can calculate the error and determine all the matches that support the estimated transformation matrix (inliers). A match is considered an inlier, if the Euclidean distance of the transformed point

to the respective point in the target space is smaller than the `inlier_threshold`. You repeat these steps in a loop until the termination condition is met. For more details on the termination condition refer to the lecture slides. Make sure to store the best error and the best inliers at each step.

3. After the loop, you should refine the projection matrix by using all inliers using the least-squares algorithm. Your `find_homography_leastsquares()` function should already support this step. Make sure to update your error and the inliers.

Complete the code for the function `find_homography_ransac` in **find_homography.py**.

Forbidden commands

`cv2.cornerHarris()`, `cv2.cornerSubPix()`, `cv2.findHomography()`

Useful commands

- `enumerate` - Return an enumerate object which consists of tuples with the count and values. Useful for `for` loops if you need the value and the index of items in a list or array.
- `cv2.getGaussianKernel` - Return the $ksize \times 1$ matrix of Gaussian filter coefficients
- `cv2.KeyPoint` - Data structure for point features. The keypoint is characterized by the 2D position, scale (proportional to the diameter of the neighborhood that needs to be taken into account), orientation and some other parameters.
- `cv2.DMatch` - Data structure for storing Matches. The `queryIdx` refers to the set of features which are provided as the first argument in the `match` function and `trainIdx` to the second argument.
- `np.gradient` - Return a set of gradients of an N-dimensional array corresponding to the derivatives with respect to each dimension
- `np.reshape` - Return an array containing the same data with a new shape.
- `np.argwhere` - Find the indices of array elements that are non-zero
- `np.r_` - Concatenate along first axis
- `np.c_` - Concatenate along second axis
- `np.transpose` or `.T` - Returns a view of the array with axes transposed
- `np.matmul` or `@`: Matrix multiplication
- `np.dot` - Return the dot product of two arrays. If both arrays are 1-D arrays, it is the inner product of vectors.
- `np.linalg.norm` - Return one of eight different matrix norms, or one of an infinite number of vector norms
- `np.linalg.lstsq` - Return the least-squares solution to a linear matrix equation

Part 2: Documentation (6 points)

Perform the following experiments with the given images and **at least one picture set you took or found yourself**, and answer the questions in a few sentences.

- Answer each numbered question separately, with the answer labelled according to the experiment and question number. For example, Experiment 1 Question 2 should be numbered 1.2. You do not need to include the question text.
- Include images to support your answers. These should be referenced in the answers. The document should include information about what parameter settings were used to generate images so that they can be reproduced. Feel free to use your own images but if you do so, upload the original image as well so we can reproduce your results.
- Your answers should attempt an explanation of why an effect occurs, not just describe what you see in the supporting images.
- Be precise. "It is better" or similarly vague statements without any additional information are not sufficient.

Points may be deducted if your report does not satisfy the above points. Only change the parameters in the file main_... in the marked places. We recommend creating a separate Python file for your experiments to not mess up the main file. If you do so, also upload this file because it makes reproducing your experiment results easier for the tutors.

Maximum 1500 words, arbitrary number of pages for result images (referenced in the text). Violating the text limit will lead to point deductions!

Honor Code

Add the honor code to the beginning of the documentation:

For the assignment I followed the honor code:

- Rule 1: Don't look at solutions or code of other students; everything you submit should be your own work
- Rule 2: Don't share your solution code with others; however, discussing ideas or general strategies is fine and encouraged
- Rule 3: If you use code from somewhere you need to cite the source directly in your code (e.g., StackOverflow). If the source is a generative AI tool (e.g., ChatGPT) provide a link to the specific Chat.

I know that I have full responsibility for what I submit. I made sure the assignment allows the teaching staff to evaluate which competences I have acquired. I am aware that plagiarism is prohibited and may lead to failure of this course.

Experiment 1 (2 points)

Investigate the Harris corner detector for rotation and scale invariance.

1. The Harris corner detector is rotation invariant. Test this, by rotating images and comparing the resulting corners before and after rotation. You can use the `rotate_bound` function from `helper_functions.py` to rotate the image. Are the same corners detected after rotation? If not, why? Suggest methods to improve rotational invariance if needed.
2. The Harris detector is not scale invariant. Give an example of an image patch which would fire the detector at one scale, but not at another. How does not being scale invariant limit the effectiveness of a detector? Use a simple example to explain.

Experiment 2 (2 points)

Match the patch descriptors of several similar images using different parameter settings. **Use at least one picture set you took yourself or found on the internet. Include these images in the submission.** Also try with non-sequential images, e.g. use desk00 and desk03.

1. How does changing the patch size of the descriptors affect the accuracy of the matching?
2. Experiment with image stitching using the Harris corner detector we implemented and a set of your own images with **main_stitching.py**. Which properties of the images and camera pose can influence the performance of the image stitching? How can the descriptors be improved to avoid those problems?

Experiment 3 (2 points)

1. How well does estimating the homography with least-squares **without** RANSAC work for the object detection problem? Can you fine-tune the Lowe ratio in `filter_matches` to get better results? Briefly discuss the reasons for the performance.
2. How does changing the RANSAC parameters `confidence` and `inlier_threshold` influence the results and the number of iterations needed?

Assistance

Please keep to the following hierarchy whenever there are questions or problems:

1. Forum: Use the TUWEL forum to discuss your problems.
2. Email for questions: machinevision@acin.tuwien.ac.at

Upload

Please upload all Python files and your documentation (pdf-format) **as one ZIP-file** via TUWEL.